

Ex.no: 09

ROUTING EXERCISE

Date :26.04.2025

Aim:

To implement React Programs on Routing

1.Real-Time Job Board Dashboard with Applications Tracking

Create a Real-Time Job Board Dashboard for an online job platform. The system should:

- **Track Job Postings:** Display new job postings as they are added in real-time.
- **Monitor Applications:** Track the number of applications per job posting and update in real-time.
- **Show Stats:** Display the total number of job posts, applications, and successful placements.
- **Visualize Data:** Use bar or line charts to show the number of applications per position over time.
- **Alert System:** Notify job seekers and recruiters of new job openings or application status changes.

CODE:

[server.js](#)

// server.js

```
import express from "express";
import http from "http";
import { Server } from "socket.io";
import cors from "cors";

const app = express();
app.use(cors());
app.use(express.json());

const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: "*" },
});

// Fake in-memory DB
let jobs = [];
let applications = [];

// API: get stats
app.get("/stats", (req, res) => {
  const stats = {
    totalJobs: jobs.length,
    totalApplications: applications.length,
    totalPlacements: applications.filter((a) => a.status === "accepted").length,
  };
  res.json(stats);
});
```

```

});

// API: post a job
app.post("/jobs", (req, res) => {
  const job = { id: Date.now(), ...req.body, applications: [] };
  jobs.push(job);
  io.emit("job_posted", job);
  res.json(job);
});

// API: apply for a job
app.post("/applications", (req, res) => {
  const appData = { id: Date.now(), ...req.body, status: "applied" };
  applications.push(appData);

  const job = jobs.find((j) => j.id === appData.jobId);
  if (job) job.applications.push(appData);

  io.emit("application_added", appData);
  res.json(appData);
});

// API: update application status
app.patch("/applications/:id", (req, res) => {
  const appId = parseInt(req.params.id);
  const { status } = req.body;

  const appIndex = applications.findIndex((a) => a.id === appId);
  if (appIndex !== -1) {
    applications[appIndex].status = status;
    io.emit("status_changed", applications[appIndex]);
    res.json(applications[appIndex]);
  } else {
    res.status(404).json({ message: "Application not found" });
  }
});

server.listen(4000, () =>
  console.log("🚀 Backend running on http://localhost:4000")
);

```

App.jsx:

```

// src/App.jsx

import { useState, useEffect } from "react";
import { useDispatch, useSelector } from "react-redux";
import io from "socket.io-client";
import axios from "axios";
import { addJob, updateJobApplications } from "../jobSlice";
import { addApplication } from "../applicationSlice";
import { ToastContainer, toast } from "react-toastify";
import {

```

```

BarChart,
Bar,
XAxis,
YAxis,
CartesianGrid,
Tooltip,
} from "recharts";
import "react-toastify/dist/ReactToastify.css";

// connect socket
const socket = io("http://localhost:4000", {
  transports: ["websocket"], // force websocket
});

export default function App() {
  const dispatch = useDispatch();
  const jobs = useSelector((state) => state.jobs);
  const applications = useSelector((state) => state.applications);

  const [stats, setStats] = useState({
    totalJobs: 0,
    totalApplications: 0,
    totalPlacements: 0,
  });

  const [jobTitle, setJobTitle] = useState("");

  // socket events
  useEffect(() => {
    socket.on("connect", () => {
      console.log("Connected to socket:", socket.id);
    });

    socket.on("job_posted", (job) => {
      dispatch(addJob(job));
      toast.info(`New Job Posted: ${job.title}`);
      setStats((prev) => ({ ...prev, totalJobs: prev.totalJobs + 1 }));
    });

    socket.on("application_added", (application) => {
      dispatch(
        updateJobApplications({ jobId: application.jobId, application })
      );
      dispatch(addApplication(application));
      toast.success(`New Application for Job ID: ${application.jobId}`);
      setStats((prev) => ({
        ...prev,
        totalApplications: prev.totalApplications + 1,
      }));
    });

    socket.on("status_changed", (application) => {
      toast.info(
        `Application ${application.id} status changed to ${application.status}`
      );
    });
  }, [socket]);
}

```

```

    if (application.status === "accepted") {
      setStats((prev) => ({
        ...prev,
        totalPlacements: prev.totalPlacements + 1,
      }));
    }
  });

  return () => {
    socket.off("connect");
    socket.off("job_posted");
    socket.off("application_added");
    socket.off("status_changed");
  };
}, [dispatch]);

// fetch stats initially
useEffect(() => {
  const fetchStats = async () => {
    try {
      const res = await axios.get("http://localhost:4000/stats");
      setStats(res.data);
    } catch (err) {
      console.error("Error fetching stats:", err);
    }
  };
  fetchStats();
}, []);

// post a job
const createJob = async () => {
  if (!jobTitle) return;
  try {
    await axios.post("http://localhost:4000/jobs", {
      title: jobTitle,
      company: "MyCompany",
    });
    setJobTitle("");
  } catch (err) {
    console.error("Error posting job:", err);
  }
};

// apply for a job
const applyForJob = async (jobId) => {
  try {
    await axios.post("http://localhost:4000/applications", {
      jobId,
      userId: Date.now(),
    });
  } catch (err) {
    console.error("Error applying:", err);
  }
};

```

```

// chart data
const chartData = jobs.map((job) => ({
  name: job.title,
  applications: job.applications?.length || 0,
}));

return (
  <div style={{ padding: 20 }}>
    <h1>Real-Time Job Board Dashboard</h1>

    <h2>Stats</h2>
    <p>Total Jobs: {stats.totalJobs}</p>
    <p>Total Applications: {stats.totalApplications}</p>
    <p>Total Placements: {stats.totalPlacements}</p>

    {/* Job Form */}
    <h2>Post a Job</h2>
    <input
      type="text"
      placeholder="Job title"
      value={jobTitle}
      onChange={(e) => setJobTitle(e.target.value)}
    />
    <button onClick={createJob}>Add Job</button>

    {/* Jobs List */}
    <h2>Jobs</h2>
    <ul>
      {jobs.map((job) => (
        <li key={job.id}>
          {job.title} — Applications: {job.applications?.length || 0}
          <button onClick={() => applyForJob(job.id)}>Apply</button>
        </li>
      ))}
    </ul>

    {/* Chart */}
    <h2>Applications Chart</h2>
    <BarChart width={500} height={300} data={chartData}>
      <CartesianGrid strokeDasharray="3 3" />
      <XAxis dataKey="name" />
      <YAxis />
      <Tooltip />
      <Bar dataKey="applications" fill="#8884d8" />
    </BarChart>

    <ToastContainer />
  </div>
);
}
jobSlice.jsx
// jobSlice.js
import { createSlice } from "@reduxjs/toolkit";

const jobSlice = createSlice({

```

```

name: "jobs",
initialState: [],
reducers: {
  addJob: (state, action) => {
    state.push(action.payload);
  },
  updateJobApplications: (state, action) => {
    const { jobId, application } = action.payload;
    const job = state.find((j) => j.id === jobId);
    if (job) {
      if (!job.applications) job.applications = [];
      job.applications.push(application);
    }
  },
},
});

export const { addJob, updateJobApplications } = jobSlice.actions;
export default jobSlice.reducer;

```

[applicationSlice.js](#)

```

// applicationSlice.js
import { createSlice } from "@reduxjs/toolkit";

const applicationSlice = createSlice({
  name: "applications",
  initialState: [],
  reducers: {
    addApplication: (state, action) => {
      state.push(action.payload);
    },
  },
});

export const { addApplication } = applicationSlice.actions;
export default applicationSlice.reducer;

```

Store.jsx

```

// store.js
import { configureStore } from "@reduxjs/toolkit";
import jobReducer from "./jobSlice";
import applicationReducer from "./applicationSlice";

const store = configureStore({
  reducer: {
    jobs: jobReducer,
    applications: applicationReducer,
  },
});

export default store;

```

OUTPUT :

Vasanth Sankar - 2312044

Real-Time Job Board Dashboard

Stats

Total Jobs: 3

Total Applications: 6

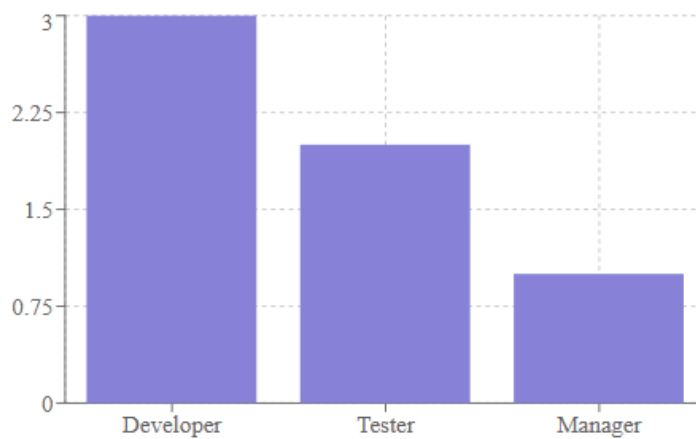
Total Placements: 0

Post a Job

Jobs

- Developer — Applications: 3
- Tester — Applications: 2
- Manager — Applications: 1

Applications Chart



2.Real-Time Restaurant Reservation and Table Availability Dashboard

Develop a Real-Time Restaurant Reservation and Table Availability Dashboard for managing restaurant reservations. The system should:

- **Track Reservations:** Monitor incoming reservation requests in real time.
- **Display Table Availability:** Show the real-time status of available, booked, and pending tables.
- **Monitor Waitlist:** Track the number of people waiting for a table and their estimated wait times.
- **Real-Time Stats:** Display the total number of reservations, cancellations, and available seats at any given time.
- **Alert System:** Send notifications to staff when a table becomes available or a reservation is approaching.

CODE:

```
`import React, { useState } from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function Navbar() {
  return (
    <nav style={{ padding: "10px", background: "#eee" }}>
      <Link to="/" style={{ margin: "0 10px" }}>Dashboard</Link>
      <Link to="/reservations" style={{ margin: "0 10px" }}>Reservations</Link>
      <Link to="/tables" style={{ margin: "0 10px" }}>Tables</Link>
      <Link to="/waitlist" style={{ margin: "0 10px" }}>Waitlist</Link>
    </nav>
  );
}

function Dashboard({ stats }) {
  return (
    <div style={{ padding: "20px" }}>
      <h2>Dashboard</h2>
      <p>Total Reservations: {stats.reservations}</p>
      <p>Cancellations: {stats.cancellations}</p>
      <p>Available Seats: {stats.seats}</p>
    </div>
  );
}

function Reservations({ reservations, addReservation }) {
  const [name, setName] = useState("");
  const [size, setSize] = useState(2);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!name) return alert("Enter a name");
    addReservation(name, size);
  }
}
```

```

    setName("");
    setSize(2);
  };

  return (
    <div style={{ padding: "20px" }}>
      <h2>Reservations</h2>

      <form onSubmit={handleSubmit} style={{ marginBottom: "20px" }}>
        <input
          type="text"
          placeholder="Customer Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          style={{ marginRight: "10px" }}
        />
        <input
          type="number"
          min="1"
          value={size}
          onChange={(e) => setSize(Number(e.target.value))}
          style={{ marginRight: "10px", width: "60px" }}
        />
        <button type="submit">Add Reservation</button>
      </form>

      <ul>
        {reservations.map((r) => (
          <li key={r.id}>
            {r.name} - Party of {r.size} ({r.status})
          </li>
        ))}
      </ul>
    </div>
  );
}

```

```

function Tables({ tables }) {
  return (
    <div style={{ padding: "20px" }}>
      <h2>Tables</h2>
      <ul>
        {tables.map((t) => (
          <li key={t.id}>
            Table {t.id} - {t.state} (capacity {t.capacity})
          </li>
        ))}
      </ul>
    </div>
  );
}

```

```

    </div>
  );
}

function Waitlist({ waitlist }) {
  return (
    <div style={{ padding: "20px" }}>
      <h2>Waitlist</h2>
      <ul>
        {waitlist.map((w) => (
          <li key={w.id}>
            {w.name} - Party of {w.size} (waiting {w.minutes} mins)
          </li>
        ))}
      </ul>
    </div>
  );
}

```

```

export default function App() {
  const [stats, setStats] = useState({
    reservations: 2,
    cancellations: 1,
    seats: 12,
  });

  const [reservations, setReservations] = useState([
    { id: 1, name: "Rahul", size: 2, status: "confirmed" },
    { id: 2, name: "Ram", size: 4, status: "pending" },
  ]);

  const [tables] = useState([
    { id: 1, state: "available", capacity: 2 },
    { id: 2, state: "occupied", capacity: 4 },
  ]);

  const [waitlist] = useState([
    { id: 1, name: "Sam", size: 3, minutes: 10 },
  ]);

  // Function to add reservation
  const addReservation = (name, size) => {
    const newRes = {
      id: reservations.length + 1,
      name,
      size,
      status: "pending",
    };
  };

```

```

setReservations([...reservations, newRes]);
setStats((prev) => ({
  ...prev,
  reservations: prev.reservations + 1,
}));
};

return (
  <BrowserRouter>
    <Navbar />
    <Routes>
      <Route path="/" element={<Dashboard stats={stats} />} />
      <Route
        path="/reservations"
        element={
          <Reservations
            reservations={reservations}
            addReservation={addReservation}
          />
        }
      />
      <Route path="/tables" element={<Tables tables={tables} />} />
      <Route path="/waitlist" element={<Waitlist waitlist={waitlist} />} />
    </Routes>
  </BrowserRouter>
);
}

```

OUTPUT:

[Dashboard](#)
[Reservations](#)
[Tables](#)
[Waitlist](#)

Vsanth Sankar - 2312044

Dashboard

Total Reservations: 2

Cancellations: 1

Available Seats: 12

[Dashboard](#) [Reservations](#) [Tables](#) [Waitlist](#)

Reservations

- Rahul - Party of 2 (confirmed)
- Ram - Party of 4 (pending)
- vasanth - Party of 2 (pending)

3.Real-Time Customer Support Chat Dashboard

Design a Real-Time Customer Support Chat Dashboard for handling live customer support queries. The system should:

- **Track Support Requests:** Monitor incoming chat requests from customers in real time.
- **Assign Support Agents:** Automatically assign support tickets to available agents or show the number of pending requests.
- **Real-Time Stats:** Display metrics like response time, number of tickets resolved, and average support duration.
- **Visualize Data:** Use charts to show the number of open and closed tickets, average wait times, and agent performance.
- **Alert System:** Notify agents when a new high-priority support request is received or when response times exceed predefined limits.

CODE:

[server.js](#)

```
import express from "express";
import http from "http";
import { Server } from "socket.io";
import cors from "cors";

const app = express();
app.use(cors());
app.use(express.json());

const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "*" } });

// In-memory storage
let tickets = [];
// ticket = { id, customerName, message, status: "open"|"closed", agent, timestamp }

let agents = ["Arjun", "Vikram"]; // sample agents
```

```

function emitUpdate() {
  io.emit("update", tickets);
}

// Create new ticket
app.post("/ticket", (req, res) => {
  const { customerName, message } = req.body;
  const id = Date.now();

  // find first available agent
  const availableAgent = agents.find(
    (a) => !tickets.some((t) => t.agent === a && t.status === "open")
  );

  const ticket = {
    id,
    customerName,
    message,
    status: "open",
    agent: availableAgent || null,
    timestamp: new Date().toISOString(),
  };

  tickets.push(ticket);

  io.emit("new_ticket", ticket);
  emitUpdate();
  res.json(ticket);
});

// Close ticket
app.post("/ticket/:id/close", (req, res) => {
  const id = parseInt(req.params.id);
  const ticket = tickets.find((t) => t.id === id);
  if (!ticket) return res.status(404).json({ message: "Ticket not found" });

  ticket.status = "closed";

  io.emit("ticket_closed", ticket);
  emitUpdate();
  res.json(ticket);
});

// Socket.io connection
io.on("connection", (socket) => {
  console.log("Client connected:", socket.id);
  socket.emit("update", tickets);
});

```

```
});

server.listen(4002, () =>
  console.log("✅ Backend running on http://localhost:4002")
);
```

App.jsx

```
import React, { useEffect, useState } from "react";
import io from "socket.io-client";
import axios from "axios";

const socket = io("http://localhost:4002");

export default function App() {
  const [tickets, setTickets] = useState([]);
  const [customerName, setCustomerName] = useState("");
  const [message, setMessage] = useState("");

  useEffect(() => {
    socket.on("update", setTickets);
    socket.on("new_ticket", (t) =>
      alert(📧 New ticket from ${t.customerName}`)
    );
    socket.on("ticket_closed", (t) =>
      alert(✅ Ticket closed: ${t.customerName}`)
    );

    return () => socket.off();
  }, []);

  const createTicket = async () => {
    if (!customerName || !message) return;
    await axios.post("http://localhost:4002/ticket", { customerName, message });
    setCustomerName("");
    setMessage("");
  };

  const closeTicket = async (id) => {
    await axios.post(`http://localhost:4002/ticket/${id}/close`);
  };

  // Stats
  const total = tickets.length;
  const open = tickets.filter((t) => t.status === "open").length;
  const closed = tickets.filter((t) => t.status === "closed").length;

  return (
    <div style={{ padding: 20, fontFamily: "sans-serif" }}>
```

```

<h1><img alt="Customer Support Dashboard icon" data-bbox="188 88 212 102"/> Customer Support Dashboard</h1>

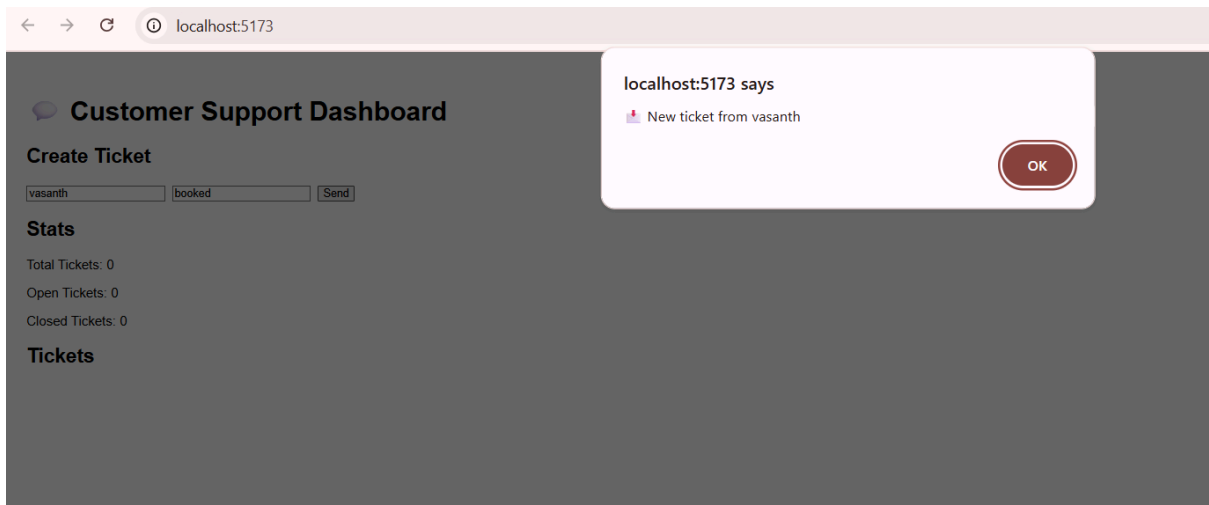
<h2>Create Ticket</h2>
<input
  placeholder="Customer Name"
  value={customerName}
  onChange={(e) => setCustomerName(e.target.value)}
/>
<input
  placeholder="Message"
  value={message}
  onChange={(e) => setMessage(e.target.value)}
  style={{ marginLeft: 8 }}
/>
<button onClick={createTicket} style={{ marginLeft: 8 }}>
  Send
</button>

<h2>Stats</h2>
<p>Total Tickets: {total}</p>
<p>Open Tickets: {open}</p>
<p>Closed Tickets: {closed}</p>

<h2>Tickets</h2>
<ul>
  {tickets.map((t) => (
    <li key={t.id}>
      {t.customerName} — {t.message} — {t.status} — Agent: {" "}
      {t.agent || "unassigned"}
      {t.status === "open" && (
        <button
          onClick={() => closeTicket(t.id)}
          style={{ marginLeft: 8 }}
        >
          Close
        </button>
      )}
    </li>
  ))}
</ul>
</div>
);
}

```

OUTPUT:



Customer Support Dashboard

Create Ticket

Stats

Total Tickets: 2

Open Tickets: 2

Closed Tickets: 0

Tickets

- vasanth — booked — open — Agent: Arjun
- Ram — error — open — Agent: Vikram

Rubrics :

Problem Analysis (10)	Coding & implementation (10)	Time Management (10)	Viva (10)	Total (40)

RESULT :

Thus , the program on react has been successfully executed and the output was verified